

Inside the Black Box: Extracting and Evaluating Automotive SBOMs and HBOMs

EcoCAR Innovation Challenge — Senior Design Project

Background and Objective

Modern vehicles are distributed computer systems containing roughly one hundred electronic control units (ECUs), many delivered by suppliers as stripped, statically linked firmware without source code. These binaries may contain inherited open-source networking, cryptographic, compression, and real-time operating system components that neither the OEM nor the vehicle team directly selected. When vulnerabilities emerge, the first challenge is often determining whether the affected software is present at all—and which version is running.

UNECE R155 and R156 and ISO/SAE 21434 make manufacturers responsible for software inventory and supply-chain cybersecurity, yet most software bill of materials (SBOM) tools assume an x86 Linux host with a package manager, filesystem, and dynamic linking. This project will build a pipeline that extracts software and hardware bills of materials from automotive source and supplier ECU binaries, then measures the accuracy of those results against ground truth constructed by the team. The EcoCAR vehicle and development environment provide the test article.

Project Needs

- Authorized ECU firmware images, source artifacts, and read-only access to vehicle or bench diagnostic data
- Automotive compilers and targets such as ARM Cortex-M/Cortex-R, Infineon TriCore, Renesas RH850, or PowerPC
- Binary-analysis and SBOM tools, including Syft, cdxgen, cve-bin-tool, SPDX, and CycloneDX validators
- Bench ECU or harness access and faculty-approved procedures for firmware handling and reverse engineering

Project Scope

- **Firmware extraction** — unpack flash and ODX update containers; recover component and version evidence from strings, build identifiers, symbols, compiler artifacts, fuzzy hashes, and function-level similarity.
- **Ground truth by construction** — compile known open-source components with recorded dependency closures across selected automotive architectures, optimization levels, and stripped/unstripped configurations.
- **Hardware inventory and binding** — recover ECU hardware identity from device trees, MCAL and board-support configuration, drivers, peripherals, and transceivers; bind each HBOM entry to its firmware image.
- **Vehicle corroboration** — compare extracted software and hardware versions with identifiers reported through read-only UDS queries on a bench harness and, where approved, the EcoCAR vehicle.
- **Evaluation and vulnerability use** — measure component- and version-level precision and recall, explain source-versus-binary disagreements, compare existing tools, and generate exposure-aware VEX output.

Technical Considerations

The measurement is the contribution, not merely the extractor. Static linking, stripping, inlining, link-time optimization, and dead-code elimination remove the package and function evidence conventional tools depend on. Version identification must be reported separately from component identification because an unversioned component can map to every CVE the project has ever had. The study should favor depth on one or two architectures and a bounded ECU corpus over shallow whole-vehicle coverage. Firmware acquisition and write operations are limited to authorized bench hardware; vehicle interaction is read-only, and supplier binaries remain within competition handling rules.

Deliverables

- Source- and binary-analysis toolchain producing conformant SPDX and CycloneDX SBOMs
- Reproducible, labelled ground-truth corpus with build recipes and dependency closures
- Evaluation report by architecture, optimization level, stripping, component identity, and version accuracy
- HBOM module with firmware-to-ECU binding, demonstrated end-to-end on at least one real ECU
- UDS corroboration and mismatch analysis, plus vulnerability correlation with exposure-aware VEX output
- Documentation, handoff materials, final report and presentation, and a workshop-paper draft if warranted